POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

A lightweight coefficient extractor in C

WILLIAM ZHANG
Département de Electrical Engineering

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Electrical Engineering

July 2026

© William Zhang, 2026.

POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

Ce mémoire intitulé :

A lightweight coefficient extractor in C

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Danielle DUBOIS, présidente
Sébastien VIGOT, membre et directeur de recherche
Marie COUTURE, membre et codirectrice de recherche
Jean TREMBLAY, membre
Joseph BROWN, membre externe

DEDICATION

À tous mes amis du labos,
vous me manquerez...

ACKNOWLEDGEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE OF CONTENTS

DEDICATION iii

ACKNOWLEDGEMENTS iv

RÉSUMÉ v

ABSTRACT vi

LIST OF TABLES ix

LIST OF FIGURES x

LIST OF SYMBOLS AND ACRONYMS xi

LIST OF APPENDICES xii

CHAPTER 1 INTRODUCTION 1
- 1.1 Definitions and basic concepts 1
- 1.2 Problem statement 1
- 1.3 Research objectives 1
- 1.4 Thesis outline 1

CHAPTER 2 LITERATURE REVIEW 2
- 2.1 Historical Background 2
- 2.2 Related work 3
  - 2.2.1 `JuMP` and `MathOptInterface` 4
  - 2.2.2 `CVXPY` 5
- 2.3 Canonicalization and Coefficient Extraction 6
  - 2.3.1 A worked example: least-squares 6
  - 2.3.2 From atoms to coefficients 7
- 2.4 Summary 8

CHAPTER 3 Setting the stage 10

CHAPTER 4 Handling parameters 15

CHAPTER 5 Benchmark results 20

CHAPTER 6 CONCLUSION 21
6.1 Synthèse des travaux / Summary of Works 21
6.2 Limitations de la solution proposée / Limitations 21
6.3 Améliorations futures / Future Research 21

REFERENCES 22

APPENDICES 25

LIST OF TABLES

LIST OF FIGURES

Figure 3.1 Expression tree for the affine expression `sum(A @ x - b) + C @ y`. . 11

Figure 3.2 Coefficient matrix accumulation for `sum(A @ x)`. Each node composes its operator with the child's coefficient matrix via sparse matrix multiplication. 13

Figure 3.3 CSR representation before and after applying `sum`. The 3-row matrix collapses to 1 row by setting `indptr` $= [0, 6]$. Duplicate column indices (0 and 2) represent values that sum together. The `indices` and `data` arrays remain unchanged—no matrix multiplication required. 14

Figure 4.1 Tensor contraction: the parameter vector weights each slice of the tensor, and the weighted sum produces the coefficient matrix $A(\theta)$. . . . 18

LIST OF SYMBOLS AND ACRONYMS

IETF	Internet Engineering Task Force
OSI	Open Systems Interconnection

LIST OF APPENDICES

Appendix A Démo 25
Appendix B Encore une annexe / Another Appendix 26
Appendix C Une dernière annexe / The Last Appendix 27

CHAPTER 1 INTRODUCTION

1.1 Definitions and basic concepts

1.2 Problem statement

1.3 Research objectives

1.4 Thesis outline

CHAPTER 2 LITERATURE REVIEW

2.1 Historical Background

Solving an optimization problem on a computer requires translating the mathematical formulation into a data format that solvers can process. For linear programs, this means constructing the constraint matrix A, the right-hand side vector b, and the objective coefficients c that define the *standard form*,

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned} \tag{2.1}$$

Manual encoding In the early days of linear programming, analysts manually computed and encoded each coefficient onto punch cards. This process was tedious, error-prone and became impractical as linear programming found more and more applications in large-scale industrial problems [1].

MPS format IBM's Mathematical Programming System (MPS) file format, developed in the 1960s, established a standardized file format for representing linear programs [2]. The MPS format uses fixed-width fields designed for 80-column punch cards, with sections for rows, columns, and coefficients. William Orchard-Hays [3] provides a comprehensive history of these early mathematical programming systems, tracing their development from 1952 at RAND Corporation through their widespread adoption on mainframe computers in the late 1960s.

Matrix generators To automate the creation of MPS files, *matrix generators* emerged in the late 1960s; they are procedural programs that compute the problem coefficients through explicit loops and output the results in MPS format [4]. Commercial systems such as `MaGen` [5], `GAMMA` [6], and `DATAFORM` [7] were among the first matrix generators which were used in various domains.

A modern approach Modelers naturally think in terms of sets, indices, and algebraic expressions, not in terms of individual matrix entries. Matrix generators simplified the encoding process but still required modelers to translate their mathematical formulations into

procedural code that explicitly constructs the coefficient matrices. This suggests that the gap between the modeler's representation of the problem and the solver's required format hadn't been resolved yet [1].

Algebraic modeling languages In the late 1970s, *algebraic modeling languages* emerged as formal languages designed to accept problem descriptions that closely mirror standard mathematical notation [8]. By working directly with the modeler's representation, these languages simplify verification (the model can be inspected as written), modification (changing the problem does not require rewriting procedural code), and communication (the formulation serves as its own documentation). In the late 1980s and early 1990s, commercial algebraic modeling systems such as `GAMS` [9], `AIMMS` [10] and `AMPL` [11] became widely adopted in both academia and industry (and they are still widely used today).

2.2 Related work

In the 2000s, advances in convex optimization made it practical to solve a much broader class of problems efficiently. Solvers such as `MOSEK` [12, 13] adopted *conic optimization* as their interface—a structured subclass of convex optimization with the standard form [14]:

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & b - Ax \in \mathcal{K}, \end{array} \tag{2.2}$$

where $\mathcal{K}$ is a closed convex cone. This formulation generalizes linear programming (when $\mathcal{K}$ is the nonnegative orthant) to include second-order cone programs (SOCPs), semidefinite programs (SDPs) and more.

Embedded modeling languages In the late 2000s and early 2010s, modeling languages emerged that are embedded in general-purpose programming languages (in contrast with standalone systems like `AMPL` and `GAMS`). Examples include `YALMIP` [15] in MATLAB, `Pyomo` [16] in Python, `JuMP` [17] in Julia, and `CasADi` [18] in C++.

Disciplined convex programming In addition, there are also *domain specific languages* that implement *disciplined convex programming* (DCP), a set of composition rules that can automatically verify convexity [19, 20]. These include `CVX` [21] in MATLAB, `CVXPY` [22, 23] in Python, and `Convex.jl` [24] in Julia.

Modern open-source modeling languages At the time of writing, most modeling languages mentioned above continue to be developed and have active communities. In the remainder of the section, we focus our attention on two in particular: `JuMP` for the design ideas introduced by `MathOptInterface`, and `CVXPY` because it is the system on which the present work builds—a choice motivated by the author's previous experience as a `CVXPY` contributor.

2.2.1 `JuMP` and `MathOptInterface`

`JuMP` is an algebraic modeling language embedded in Julia, originally developed in 2012 by Iain Dunning, Joey Huchette, and Miles Lubin at the MIT Operations Research Center [17, 25]. Its abstraction layer, `MathOptInterface` (`MOI`) [26], rethinks how a modeling language communicates with solvers, moving away from the traditional approach of reducing every problem to a single standard form.

Function-set pairs `MOI` represents each constraint as a *function-set pair*: a mathematical function paired with the set it must belong to. For example, a linear inequality $a^T x \leq b$ is encoded as (`ScalarAffineFunction`, `LessThan(b)`), while a second-order cone constraint $\|Ax+b\|_2 \leq c^T x+d$ uses (`VectorAffineFunction`, `SecondOrderCone`). This is more granular than committing to a single conic standard form as in (2.2)—`MOI` preserves the semantic type of each constraint. Solvers that accept specific constraint types can receive them directly, without unnecessary reformulations.

Bridges When a solver does not natively support a particular function-set pair, `MOI` uses *bridges*—small, composable transformations between equivalent constraint representations. For instance, a bridge can split an interval constraint into separate less-than and greater-than constraints. The bridge system maintains a directed hyper-graph of all known transformations and finds the shortest path to the target solver's supported types. Adding a reformulation once in `MOI` benefits all solvers, rather than requiring each solver wrapper to implement its own conversions.

Incremental problem construction Some solvers, notably `MOSEK` [12] and `HiGHS` [27], expose APIs that add constraints one at a time. `MOI` supports this incremental mode natively: rather than assembling a complete coefficient matrix and passing it to the solver in one call, `MOI` can issue individual `add_constraint` calls as the model is built. For example, adding a cut $a^T x \leq b$ after an initial solve requires only one such call, not a full reconstruc-

tion of the problem data. This is particularly useful in algorithms that iteratively refine a formulation, such as cutting-plane methods or branch-and-cut, where constraints are added between successive solves.

Extensibility `MOI` makes it straightforward to support new constraint types. A solver declares which function-set pairs it handles, and new bridges can be registered at any time. `Hypatia` [28] [29–31].

2.2.2 `CVXPY`

`CVXPY` is a domain-specific language for convex optimization embedded in Python, that was originally developed by Steven Diamond and Stephen Boyd at Stanford University [22]. Akshay Agrawal later redesigned the rewriting system in `CVXPY` [23], which enabled support for other disciplined languages [32–34], and the project has since grown through contributions from many others. Unlike `MOI`, which preserves constraint diversity, `CVXPY` channels all problems through a single compilation pipeline targeting one unified standard form.

Unified standard form `CVXPY` rewrites every problem into a conic standard form with a quadratic objective:

$$\begin{aligned} &\text{minimize} && \tfrac{1}{2}\, x^T P\, x + c^T x \\ &\text{subject to} && Ax + b \in \mathcal{K}, \end{aligned} \tag{2.3}$$

Because the rewriting system targets a unique form, solver interfaces only need to map from this single representation. This centralization simplifies the codebase: one set of rewriting rules rather than a combinatorial number of bridges, and every solver benefits from the same transformation pipeline.

Disciplined convex programming `CVXPY` implements DCP rules that automatically verify convexity and transform high-level expressions into conic form. A user can write `cp.sum_squares(A @ x - b)` and `CVXPY` will verify convexity, introduce auxiliary variables, and emit the equivalent second-order cone constraint transparently. In contrast, `JuMP` requires users to formulate conic constraints directly (*e.g.*specifying a rotated second-order cone rather than a sum-of-squares objective), demanding deeper conic optimization knowledge.

Parameters and efficient re-solving `CVXPY` provides first-class support for parameters via `cp.Parameter`. When a problem contains parameters, `CVXPY` compiles the structure once—sparsity pattern, cone types, variable ordering—and stores a mapping from parameter

values to numerical coefficients. Subsequent solves only require evaluating this mapping and calling the solver, skipping the full canonicalization. The `cvxpygen` library [35] extends this by generating standalone C code embedding the compiled structure, enabling deployment without a Python runtime.

2.3 Canonicalization and Coefficient Extraction

The previous sections described what modeling languages do at a high level: they accept mathematical formulations and produce solver-ready data. This section examines *how* that translation works, using a concrete example to illustrate the canonicalization pipelines of both `MOI` and `CVXPY`.

2.3.1 A worked example: least-squares

Consider the least-squares problem

$$\text{minimize} \quad \|Ax - b\|_2^2, \tag{2.4}$$

with data $A \in \mathbf{R}^{2\times 2}$ and $b \in \mathbf{R}^2$. This problem is small enough to trace every step of the canonicalization, yet it exercises the key mechanisms of both systems.

`MOI`'s bridge-based path In `JuMP`, writing `@objective(model, Min, sum((A*x - b).^2))` creates a *scalar quadratic function*. `MOI` represents the objective as a function-set pair (`ScalarQuadraticFunction`, `MinSense`). If the target solver accepts quadratic objectives directly (*e.g.*`MOSEK`, `Gurobi`), no further transformation is needed—`MOI` passes the quadratic coefficients as-is.

If the solver only supports conic constraints, `MOI`'s bridge system activates. A *quadratic-to-SOC bridge* rewrites the quadratic objective into a second-order cone constraint by introducing an epigraph variable t and enforcing $\|Ax - b\|_2 \leq t$, with the objective changed to minimize t. Each bridge is a small, self-contained transformation; `MOI` composes as many as needed to reach the solver's supported types.

`CVXPY`'s compilation path In `CVXPY`, the same problem is written as

```
x = cp.Variable(2)
prob = cp.Problem(cp.Minimize(cp.sum_squares(A @ x - b)))
```

`CVXPY` processes this through several stages:

1. **Expression tree.** `CVXPY` builds a tree whose root is `sum_squares`, with children `sub`$(-)$, which in turn has children `matmul`(A, x) and constant b.

2. **DCP verification.** The `sum_squares` atom is convex, and its argument $Ax - b$ is affine; by DCP composition rules, the overall expression is convex.

3. **Epigraph reformulation.** The `sum_squares` atom's canonicalization method introduces an auxiliary variable $t \in \mathbf{R}$ and rewrites the objective as minimize t subject to $\|Ax - b\|_2^2 \leq t$, which is equivalent to $\left\| \begin{smallmatrix} Ax-b \\ \frac{1}{2}(t-1) \end{smallmatrix} \right\|_2 \leq \frac{1}{2}(t+1)$—a second-order cone constraint.

4. **Conic standard form.** `CVXPY` assembles the constraint into the form $\mathcal{A}z + \beta \in \mathcal{K}$ where $z = (x, t)$, and $\mathcal{K}$ is a second-order cone. The objective becomes $c^T z$ with $c = (0, 0, 1)$, selecting the epigraph variable.

For our 2×2 example with $z = (x_1, x_2, t)^T$, the final solver-ready data is:

$$c = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad \mathcal{A} = \begin{pmatrix} 0 & 0 & \frac{1}{2} \\ 0 & 0 & -\frac{1}{2} \\ a_{11} & a_{12} & 0 \\ a_{21} & a_{22} & 0 \end{pmatrix}, \quad \beta = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \\ -b_1 \\ -b_2 \end{pmatrix}, \tag{2.5}$$

with $\mathcal{K} = \mathcal{Q}^4$ (second-order cone of dimension 4). The matrices $\mathcal{A}$, β, and c are the *coefficient data* that this thesis is concerned with extracting.

2.3.2 From atoms to coefficients

The least-squares example illustrates a general pattern. Every `CVXPY` problem goes through the same pipeline, regardless of complexity.

What is coefficient extraction? Coefficient extraction is the process of transforming a high-level convex optimization formulation into the numerical arrays that a solver consumes—the matrices P, $\mathcal{A}$, vectors c, β, and cone specifications $\mathcal{K}$ of the standard form (2.3). This is the final, computational step of the canonicalization pipeline: once the problem has been rewritten into conic form, the system must traverse the resulting expression trees and assemble concrete numerical data.

The canonicalization pipeline `CVXPY`'s full pipeline proceeds in three phases:

1. **DCP verification.** Each node in the expression tree is checked against DCP composition rules. If the overall problem is verified as convex (or concave for maximization), canonicalization proceeds; otherwise, `CVXPY` raises an error.

2. **Atom canonicalization.** Each non-affine atom (*e.g.*`sum_squares`, `norm`, `log`) is replaced by its conic equivalent. This step introduces auxiliary variables and constraints, producing a reformulated problem where every expression is affine and every constraint has the form $\mathcal{A}z + \beta \in \mathcal{K}$.

3. **Coefficient assembly.** The system traverses the affine expression trees that remain after atom canonicalization, evaluating each linear operator node and assembling the results into sparse coefficient matrices. Chapter 3 details how these linear operators are represented and composed.

Parametric compilation When a problem contains `cp.Parameter` objects, `CVXPY` can separate structure from data. The sparsity pattern and cone types are determined once at compilation time; only the numerical values in $\mathcal{A}$ and β change when parameters are updated. This separation is what makes it possible to extract the coefficient computation as a standalone routine—a mapping from parameter values to solver-ready arrays—which is the core idea behind the C extractor developed in this thesis.

Contrast with `MOI` `MOI`'s bridge-based architecture achieves a similar end—translating high-level formulations into solver-ready data—but through a different mechanism. Rather than compiling to a single standard form, `MOI` applies targeted transformations that preserve constraint semantics as long as possible. This design favors flexibility and incrementality (constraints can be added or removed without recompiling the entire problem), while `CVXPY`'s design favors a clean compilation model where the entire problem is reduced to one uniform representation. The trade-off is architectural: `MOI`'s approach is more modular, while `CVXPY`'s approach makes the coefficient extraction step uniform and predictable, which is advantageous for code generation.

2.4 Summary

The evolution from manual encoding to algebraic modeling languages reveals a consistent trend toward raising the abstraction level between modeler and solver. `JuMP`'s `MathOptInterface`

extends this trajectory with function-set pairs, composable bridges, and incremental construction, offering a flexible architecture that preserves constraint semantics all the way to the solver. `CVXPY` takes a complementary path: by funneling all problems through a single conic standard form and providing DCP verification and first-class parameters, it enables a compilation model where the problem structure is determined once and only numerical data changes between solves. Neither system, however, provides a self-contained C implementation of the coefficient extraction pipeline. The ability to compile a `CVXPY` problem into a lightweight, dependency-free C module that maps parameters to solver-ready coefficient matrices is the contribution of this thesis.

CHAPTER 3 Setting the stage

Linear operator tree CVXPY's canonicalization guarantees that all constraints and the objective function in are affine functions of the variable x. An affine function can be expressed as the composition of linear functions and addition of constants. CVXPY represents each affine function as an *expression tree*, where each node is a linear function or *atom*.

A first example When you specify a single variable in CVXPY, such as

```
x = cp.Variable(4)
```

the internal representation is the identity linear operator of size 4×4:

$$\begin{array}{c} \begin{matrix} x_1 & x_2 & x_3 & x_4 \end{matrix} \\ \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{array}$$

A second example When you specify a sum operator on a variable in CVXPY, such as

```
x = cp.Variable(4)
y = cp.sum(x)
```

the internal representation is a 1×4 matrix of all ones:

$$\begin{array}{c} \begin{matrix} x_1 & x_2 & x_3 & x_4 \end{matrix} \\ \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \end{array}$$

Applying this operator to the variable vector yields the sum:

$$\begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = x_1 + x_2 + x_3 + x_4$$

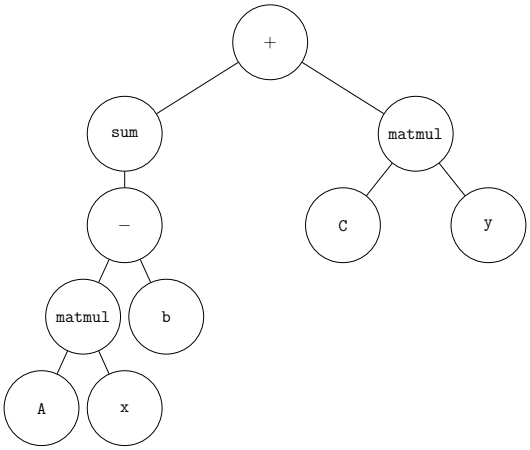


Figure 3.1 Expression tree for the affine expression `sum(A @ x - b) + C @ y`.

A matrix example When you specify a matrix variable in CVXPY, such as

```
X = cp.Variable((3, 2))
```

the entries of the matrix are vectorized in column-major order, and the internal representation is the identity linear operator of size 6×6:

$$\begin{array}{c} \begin{array}{cccccc} X_{11} & X_{21} & X_{31} & X_{12} & X_{22} & X_{32} \end{array} \\ \left[\begin{array}{cccccc} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{array}\right] \end{array}$$

A composition example When you specify a more complex linear function in CVXPY, such as

```
x = cp.Variable(3)
A = np.random.randn(5, 3)
b = np.random.randn(5)
y = cp.sum(A @ x + b)
```

the internal representation is the composition of three linear operators:

- the matrix multiplication operator $A \in \mathbf{R}^{5\times 3}$,
- the vector addition operator $+b$ where $b \in \mathbf{R}^5$,
- the sum operator $\mathbf{1}^T \in \mathbf{R}^{1\times 5}$.

The constant b is incorporated by augmenting the matrix with a column for the constant term. The resulting linear operator is $\mathbf{1}^T[A \mid b] \in \mathbf{R}^{1\times 4}$:

$$\underbrace{\begin{bmatrix}1 & 1 & 1 & 1 & 1\end{bmatrix}}_{\mathbf{1}^T}\underbrace{\left[\begin{array}{ccc|c} A_{11} & A_{12} & A_{13} & b_1 \\ A_{21} & A_{22} & A_{23} & b_2 \\ A_{31} & A_{32} & A_{33} & b_3 \\ A_{41} & A_{42} & A_{43} & b_4 \\ A_{51} & A_{52} & A_{53} & b_5 \end{array}\right]}_{[A|b]} = \begin{array}{cccc} x_1 & x_2 & x_3 & 1 \end{array}\left[\begin{array}{ccc|c} \sum_i A_{i1} & \sum_i A_{i2} & \sum_i A_{i3} & \sum_i b_i \end{array}\right]$$

A multi-variable example When you specify multiple variables and constraints in CVXPY, such as

```
x = cp.Variable()
y = cp.Variable(3)
z = cp.Variable(2)
constraints = [8*x == 28, cp.sum(y) + 5*cp.sum(z) == 58, z == 26]
```

the variables are concatenated into a single vector $[x, y_1, y_2, y_3, z_1, z_2]^T$. Each constraint is converted to the form $Ax + b = 0$, producing a linear operator with a constant column:

- `8*x == 28` becomes $8x - 28 = 0$:

$$\begin{array}{cccccc|c} x & y_1 & y_2 & y_3 & z_1 & z_2 & 1 \end{array}$$
$$\left[\begin{array}{cccccc|c} 8 & 0 & 0 & 0 & 0 & 0 & -28 \end{array}\right]$$

- `cp.sum(y) + 5*cp.sum(z) == 58` becomes $\sum y_i + 5\sum z_i - 58 = 0$:

$$\begin{array}{cccccc|c} x & y_1 & y_2 & y_3 & z_1 & z_2 & 1 \end{array}$$
$$\left[\begin{array}{cccccc|c} 0 & 1 & 1 & 1 & 5 & 5 & -58 \end{array}\right]$$

- `z == 26` becomes $z_1 - 26 = 0$ and $z_2 - 26 = 0$:

$$\begin{array}{cccccc|c} x & y_1 & y_2 & y_3 & z_1 & z_2 & 1 \\ \hline 0 & 0 & 0 & 0 & 1 & 0 & -26 \\ 0 & 0 & 0 & 0 & 0 & 1 & -26 \end{array}$$

These are then stacked into a single linear operator:

$$\begin{array}{cccccc|c} x & y_1 & y_2 & y_3 & z_1 & z_2 & 1 \\ \hline 8 & 0 & 0 & 0 & 0 & 0 & -28 \\ 0 & 1 & 1 & 1 & 5 & 5 & -58 \\ 0 & 0 & 0 & 0 & 1 & 0 & -26 \\ 0 & 0 & 0 & 0 & 0 & 1 & -26 \end{array}$$

The current approach: sparse matrix multiplication CVXPY's current approach is to convert the linear operator tree into a single sparse matrix. The tree is traversed in post-order: each leaf node returns its coefficient matrix, and each internal node composes its children's matrices via sparse matrix multiplication. For example, in the expression `sum(A @ x)`, the traversal first returns A at the `matmul` node, then multiplies by $\mathbf{1}^T$ at the `sum` node, yielding $\mathbf{1}^T A$. The final matrix at the root is returned.

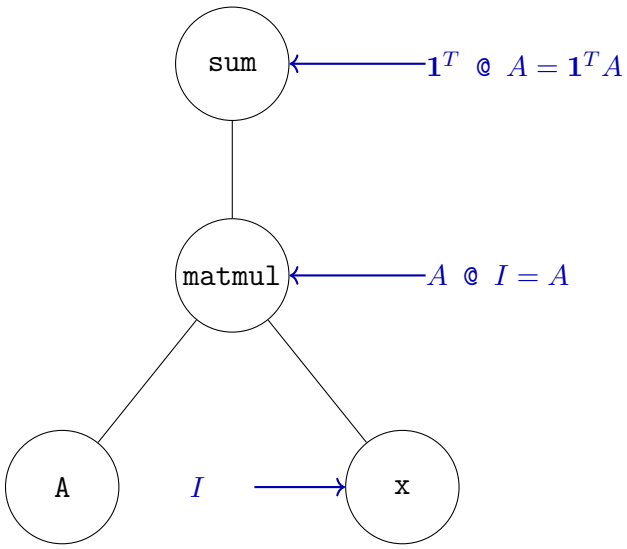


Figure 3.2 Coefficient matrix accumulation for `sum(A @ x)`. Each node composes its operator with the child's coefficient matrix via sparse matrix multiplication.

Limitations of current approach This approach requires storing intermediate sparse matrices at each node during traversal. For deep expression trees or large problem dimensions, this leads to significant memory overhead. Additionally, sparse matrix multiplication is computationally expensive and may produce matrices with more nonzeros than the operands, further increasing memory and runtime costs.

Our approach: direct encoding of linear operators We keep the coefficient matrix in CSR (Compressed Sparse Row) format and directly manipulate its index arrays without forming intermediate matrices. Each linear operator is implemented as a transformation on the CSR arrays `(indptr, indices, data)`. For example, the `sum` operator collapses all rows into row 0 by setting all entries of `indptr` to $[0, \texttt{nnz}]$, effectively mapping every row index to 0 without any matrix multiplication.

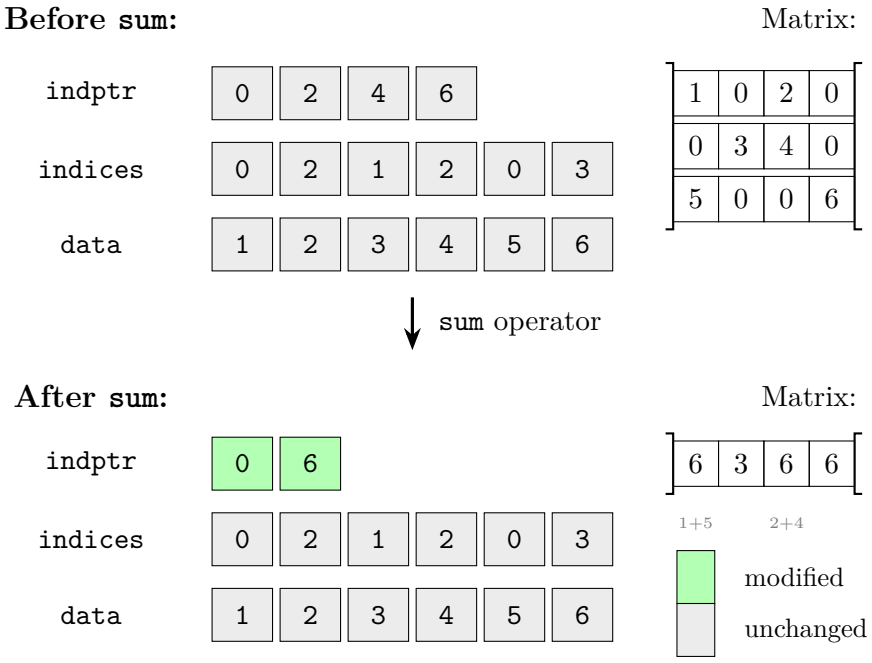


Figure 3.3 CSR representation before and after applying `sum`. The 3-row matrix collapses to 1 row by setting `indptr` $= [0, 6]$. Duplicate column indices (0 and 2) represent values that sum together. The `indices` and `data` arrays remain unchanged—no matrix multiplication required.

CHAPTER 4 Handling parameters

Parameters in optimization models Many applications require solving a sequence of related optimization problems that differ only in their data. In Model Predictive Control (MPC), a controller solves an optimization problem at each timestep with the current system state as input. In parameter sweeps, we explore the behavior of a system across a range of parameter values. The key insight is that these problems share the same structure—only the numerical coefficients change. We want to solve many such problems efficiently without reconstructing the problem from scratch each time.

A first example: scalar objective coefficient Consider the simplest case where parameters affect the objective function. The following CVXPY code defines a linear objective with a parametric coefficient vector:

```
c = cp.Parameter(3)
x = cp.Variable(3)
objective = c @ x  # c^T x
```

When the parameter vector $\theta = [c_1, c_2, c_3]^T$ is set, the objective coefficient vector $c(\theta)$ is computed. In this case, the mapping from θ to c is simply the identity:

$$c(\theta) = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{matrix} \theta_1 & \theta_2 & \theta_3 \\ \end{matrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

We can write this as $c(\theta) = M\theta$ where M is the mapping matrix. Here $M = I$, but as we will see, more complex expressions produce sparse, non-identity mapping matrices.

A second example: mixed constants and parameters Consider an objective where only some coefficients are parametric:

```
c = cp.Parameter(2)
x = cp.Variable(4)
objective = c @ x[:2] + np.array([5, 3]) @ x[2:]
```

This is equivalent to $[c_1, c_2, 5, 3]^T x$, where two coefficients are parametric and two are constant. The mapping is now:

$$c(\theta) = \begin{bmatrix} c_1 \\ c_2 \\ 5 \\ 3 \end{bmatrix} = \begin{matrix} \\ \\ \\ \\ \end{matrix} \begin{matrix} 1 & \theta_1 & \theta_2 \end{matrix} \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 5 & 0 & 0 \\ 3 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ \theta_1 \\ \theta_2 \end{bmatrix}$$

The first column represents the constant part (multiplied by the implicit constant 1), and the remaining columns map θ_1 and θ_2 to their positions. Note that this matrix is sparse—most entries are zero. This sparsity is typical: each coefficient depends on at most one parameter, so each row has few nonzeros.

Generalizing to constraint matrices: the 3D tensor So far we have considered vectors $c(\theta)$. Constraints involve matrices $A(\theta)$, where each entry A_{ij} may depend on parameters. Consider a parametric constraint matrix:

```
P = cp.Parameter((2, 2))
x = cp.Variable(2)
constraint = P @ x <= 0
```

The parameter P contributes four scalar parameters $\theta_1, \theta_2, \theta_3, \theta_4$ (in column-major order: $P_{11}, P_{21}, P_{12}, P_{22}$). The constraint matrix $A(\theta)$ equals P:

$$A(\theta) = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} \theta_1 & \theta_3 \\ \theta_2 & \theta_4 \end{bmatrix}$$

To represent how each matrix entry depends on the parameters, we use a 3D tensor. Flattening the matrix entries (i, j) into rows and the parameter indices into columns:

$$\begin{matrix} \\ A_{11} \\ A_{12} \\ A_{21} \\ A_{22} \end{matrix} \begin{matrix} 1 & \theta_1 & \theta_2 & \theta_3 & \theta_4 \end{matrix} \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Each row shows how one matrix entry is computed from the parameters. The first column (labeled 1) is for constants; here it is all zeros since P has no constant part.

A mixed example with constants When parameters are combined with constants, both contribute to the tensor:

```
P = cp.Parameter((2, 2))
Q = np.array([[1, 2], [3, 4]])
x = cp.Variable(2)
constraint = (P + Q) @ x <= 0
```

Now $A(\theta) = P + Q$, so each entry is the sum of a parameter and a constant:

$$A(\theta) = \begin{bmatrix} \theta_1 + 1 & \theta_3 + 2 \\ \theta_2 + 3 & \theta_4 + 4 \end{bmatrix}$$

The tensor representation includes the constants in the first column:

$$\begin{array}{c} \\ A_{11} \\ A_{12} \\ A_{21} \\ A_{22} \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & \theta_1 & \theta_2 & \theta_3 & \theta_4 \end{array} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 2 & 0 & 0 & 1 & 0 \\ 3 & 0 & 1 & 0 & 0 \\ 4 & 0 & 0 & 0 & 1 \end{bmatrix} \end{array}$$

This representation makes evaluation straightforward: to compute $A(\theta)$, we multiply each row by the extended parameter vector $[1, \theta_1, \theta_2, \theta_3, \theta_4]^T$.

The tensor contraction operation Given the tensor representation, evaluating $A(\theta)$ is a tensor contraction: we sum over the parameter dimension. Conceptually, think of the tensor as a stack of matrices—one "slice" for the constants and one for each parameter:

- Slice 0 (constants): $A_0 = Q$
- Slice 1 (θ_1): has a 1 at position $(1, 1)$, zeros elsewhere
- Slice 2 (θ_2): has a 1 at position $(2, 1)$, zeros elsewhere
- Slice 3 (θ_3): has a 1 at position $(1, 2)$, zeros elsewhere

- Slice 4 (θ_4): has a 1 at position $(2, 2)$, zeros elsewhere

The evaluation is:

$$A(\theta) = A_0 \cdot 1 + A_1 \cdot \theta_1 + A_2 \cdot \theta_2 + A_3 \cdot \theta_3 + A_4 \cdot \theta_4$$

This is simply a weighted sum of the slices.

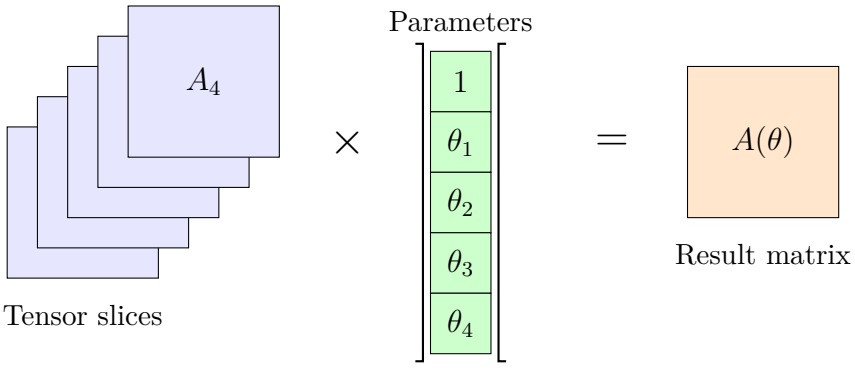


Figure 4.1 Tensor contraction: the parameter vector weights each slice of the tensor, and the weighted sum produces the coefficient matrix $A(\theta)$.

Sparse representation The tensor is extremely sparse. In the example above, each slice A_1, A_2, A_3, A_4 has exactly one nonzero entry. More generally, each parameter affects only specific entries of the coefficient matrices. We store the tensor in a sparse format:

- Shape: $(m, n, p + 1)$ where $A(\theta) \in \mathbf{R}^{m \times n}$ and $\theta \in \mathbf{R}^p$
- Storage: list of (row, column, slice, value) tuples for nonzero entries

Evaluation produces $A(\theta)$ in COO (Coordinate) format, which we then convert to CSR for efficient arithmetic.

Summary The tensor approach represents parametric coefficient matrices as a sparse 3D tensor:

- Each matrix entry's dependence on parameters is encoded as a row of the flattened tensor
- The first slice holds constants; subsequent slices hold parameter coefficients

- Evaluation is a sparse tensor-vector multiplication
- Sparsity is high because each coefficient typically depends on few parameters

This representation enables efficient re-evaluation when parameters change, without rebuilding the optimization problem structure.

CHAPTER 5 Benchmark results

CHAPTER 6 CONCLUSION

Texte / Text.

6.1 Synthèse des travaux / Summary of Works

Texte / Text.

6.2 Limitations de la solution proposée / Limitations

6.3 Améliorations futures / Future Research

Texte / Text.

REFERENCES

[1] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," 1978.

[2] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[3] W. Orchard-Hays, "History of mathematical programming systems," *Annals of the History of Computing*, vol. 6, no. 3, pp. 296–312, 1984.

[4] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[5] *MaGen Reference Manual*, Haverly Systems, Inc., Denville, NJ, 1977.

[6] *GAMMA 3.4 Programmer Reference*, Sperry Univac Computer Systems, St. Paul, MN, 1977.

[7] *MIPS III DATAFORM User Manual*, Ketron, Inc., Arlington, VA, 1975.

[8] R. Fourer, "Modeling languages versus matrix generators for linear programming," *ACM Trans. Math. Softw.*, vol. 9, no. 2, pp. 143–183, Jun. 1983. [Online]. Available: https://doi.org/10.1145/357456.357457

[9] A. Brooke, D. Kendrick, and A. Meeraus, *GAMS: A User's Guide*. Redwood City, CA: The Scientific Press, 1988.

[10] J. Bisschop, *AIMMS optimization modeling*, 2006.

[11] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[12] M. ApS, *MOSEK optimization suite 11.0*, 2026.

[13] ——, "MOSEK modeling cookbook," 2026.

[14] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[15] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE, 2004, pp. 284–289.

[16] W. E. Hart, J.-P. Watson, and D. L. Woodruff, "Pyomo: modeling and solving mathematical programs in Python," *Mathematical Programming Computation*, vol. 3, no. 3, pp. 219–260, 2011.

[17] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[18] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[19] M. Grant, "Disciplined Convex Programming," Ph.D. dissertation, Stanford University, Dec. 2004, ph.D. dissertation. [Online]. Available: https://web.stanford.edu/~boyd/papers/pdf/mcg_thesis.pdf

[20] M. Grant, S. Boyd, and Y. Ye, "Disciplined convex programming," in *Global Optimization: From Theory to Implementation*, L. Liberti and N. Maculan, Eds. New York: Springer, 2006, pp. 155–210.

[21] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," https://cvxr.com/cvx, Aug. 2012.

[22] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[23] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[24] M. Udell, K. Mohan, D. Zeng, J. Hong, S. Diamond, and S. Boyd, "Convex optimization in Julia," in *2014 first workshop for high performance technical computing in dynamic languages*. IEEE, 2014, pp. 18–28.

[25] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[26] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, "Mathoptinterface: a data structure for mathematical optimization problems," 2020. [Online]. Available: https://arxiv.org/abs/2002.03447

[27] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.

[28] C. Coey, L. Kapelevich, and J. P. Vielma, "Solving natural conic formulations with Hypatia.jl," *INFORMS Journal on Computing*, vol. 34, no. 5, pp. 2686–2699, 2022.

[29] B. O'Donoghue, E. Chu, N. Parikh, and S. Boyd, "Conic optimization via operator splitting and homogeneous self-dual embedding," *Journal of Optimization Theory and Applications*, vol. 169, no. 3, pp. 1042–1068, June 2016. [Online]. Available: http://stanford.edu/~boyd/papers/scs.html

[30] B. O'Donoghue, "Operator splitting for a homogeneous embedding of the linear complementarity problem," *SIAM Journal on Optimization*, vol. 31, pp. 1999–2023, August 2021.

[31] D. Cederberg and S. Boyd, "Projections onto spectral matrix cones," 2025. [Online]. Available: https://arxiv.org/abs/2511.01089

[32] A. Agrawal, S. Diamond, and S. Boyd, "Disciplined geometric programming," *Optimization Letters*, vol. 13, pp. 961–976, 2019.

[33] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, pp. 1643–1657, 2020.

[34] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and Z. Kolter, "Differentiable convex optimization layers," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[35] M. Schaller, G. Banjac, S. Diamond, A. Agrawal, B. Stellato, and S. Boyd, "Embedded code generation with cvxpy," *IEEE Control Systems Letters*, vol. 6, pp. 2653–2658, 2022.

APPENDIX A DÉMO

Texte de l'annexe A. Remarquez que la phrase précédente se termine par une lettre majuscule suivie d'un point. On indique explicitement cette situation à LaTeX afin que ce dernier ajuste correctement l'espacement entre le point final de la phrase et le début de la phrase suivante.

APPENDIX B ENCORE UNE ANNEXE / ANOTHER APPENDIX

Texte de l'annexe B en mode «landscape».

APPENDIX C UNE DERNIÈRE ANNEXE / THE LAST APPENDIX

Texte de l'annexe C.